

ĐẠI HỌC BÁCH KHOA HÀ NỘI
Trường Công nghệ Thông tin và Truyền thông

TÀI LIỆU KIỂM THỬ PHẦN MỀM

Phiên bản 0.0.1

HỆ THỐNG NAVIMART

Quản lý kho thực phẩm gia đình, kế hoạch bữa ăn,
công thức và danh sách mua sắm dùng chung

Môn: Phát triển phần mềm theo chuẩn kỹ năng ITSS

Họ và tên	MSSV
.....	
.....	
.....	

Hà Nội, ngày 14 tháng 6 năm 2026

Contents

1	Giới thiệu và mục đích tài liệu	3
1.1	Mục đích	3
1.2	Đối tượng sử dụng	3
1.3	Phạm vi	3
2	Định hướng ITSS và quy trình kiểm thử	4
3	Tổng quan hệ thống và phạm vi kiểm thử	4
3.1	Kiến trúc backend	4
3.2	Trong và ngoài phạm vi	4
4	Chiến lược kiểm thử	5
5	Môi trường, công cụ và dữ liệu kiểm thử	6
6	Thiết kế testcase và ma trận truy vết	7
7	Kiểm thử đơn vị	8
8	Kiểm thử API/Controller	9
9	Kiểm thử bảo mật	9
10	Kiểm thử tích hợp và đầu-cuối	10
10.1	Chuẩn hóa môi trường thiết lập	10
10.2	Case study: phát hiện và sửa lỗi trừ kho vượt mức (oversell)	10
11	Kết quả thực thi và độ phủ	11
11.1	Kết quả regression	11
11.2	Độ phủ mã nguồn backend	11
11.3	Phân tích lý do	11
12	Rủi ro, lỗi và tiêu chí đóng	12
12.1	Sổ rủi ro	12
12.2	Lỗi đã ghi nhận	12

12.3 Tiêu chí đóng P0	12
13 Đánh giá mức độ đáp ứng và kết luận	12
13.1 Đánh giá theo từng tầng	12
13.2 Kết luận	13
13.3 Lộ trình cải tiến	13
A Phụ lục A. Danh mục testcase chi tiết	13
A.1 Backend — kiểm thử đơn vị	13
A.2 Backend — kiểm thử API, đầu-cuối và controller	14
A.3 Frontend (Vitest)	14
B Phụ lục B. Checklist chạy và nghiệm thu	15

1. Giới thiệu và mục đích tài liệu

NaviMart là một ứng dụng quản lý gia đình xoay quanh thực phẩm: theo dõi kho thực phẩm trong nhà, nhắc hạn sử dụng, lập kế hoạch bữa ăn, quản lý công thức nấu ăn và tạo danh sách mua sắm dùng chung giữa các thành viên. Vì hệ thống chạm trực tiếp đến dữ liệu được nhiều người cùng chỉnh sửa — tồn kho, quyền chia sẻ, trạng thái thanh toán của danh sách mua sắm — nên độ tin cậy của các quy tắc nghiệp vụ và của lớp phân quyền là yếu tố quyết định chất lượng. Tài liệu này được biên soạn để chứng minh các quy tắc đó hoạt động đúng, đồng thời ghi nhận một cách minh bạch những phần đã được kiểm chứng và những phần còn là rủi ro.

Tài liệu không chỉ liệt kê số lượng testcase pass/fail mà đi theo định hướng *bàn giao kiểm thử*: giải thích mục tiêu của từng loại kiểm thử, công cụ và dữ liệu sử dụng, cách đánh giá kết quả, các lỗi phát hiện được cùng cách xử lý, và điều kiện để có thể ký duyệt bản giao. Cấu trúc trình bày tham chiếu khung tài liệu kiểm thử theo ISO/IEC/IEEE 29119-3, nhưng được điều chỉnh cho phù hợp với một dự án thực tế gồm **backend NestJS** và **frontend React**.

1.1 Mục đích

Công tác kiểm thử trong dự án hướng tới sáu mục tiêu. Thứ nhất, xác minh backend thực hiện đúng các nghiệp vụ trọng yếu: xác thực người dùng, quản lý gia đình và chia sẻ, kho thực phẩm, công thức, kế hoạch bữa ăn, danh sách mua sắm và báo cáo. Thứ hai, kiểm tra hợp đồng (contract) của API ở mức HTTP — bao gồm đường dẫn, phương thức, tham số, kiểm tra dữ liệu đầu vào và mã lỗi trả về. Thứ ba, kiểm tra logic nghiệp vụ và các chuyển trạng thái ở mức đơn vị. Thứ tư, kiểm tra cơ chế xác thực và phân quyền (JWT, guard theo vai trò và theo quyền gia đình). Thứ năm, kiểm tra tính bền vững của dữ liệu và các luồng nghiệp vụ xuyên suốt bằng kiểm thử đầu-cuối trên cơ sở dữ liệu thật. Cuối cùng, đo lường độ phủ mã nguồn và đề xuất lộ trình cải tiến.

1.2 Đối tượng sử dụng

Tài liệu phục vụ ba nhóm người đọc với những nhu cầu khác nhau: **lập trình viên** cần biết phạm vi test hiện có, cách chạy và bổ sung test, cũng như các lỗi cần xử lý trước khi gộp mã; **kiểm thử viên/QA** cần đánh giá thiết kế testcase, kết quả thực thi, độ phủ và bằng chứng để ký duyệt; và **quản lý dự án** cần cơ sở để ra quyết định về chất lượng bàn giao và ưu tiên xử lý rủi ro.

1.3 Phạm vi

Phạm vi của tài liệu bao gồm cả backend (mã kiểm thử trong **backend/src** và **backend/test**) và frontend (**frontend/src**, **frontend/test**), cùng các báo cáo độ phủ sinh ra từ Jest và Vitest. Đây là điểm mở rộng so với khung tham chiếu thông thường vốn chỉ giới hạn ở backend. Một số hạng mục nằm ngoài phạm vi hiện tại do chưa có hạ tầng hoặc artifact: kiểm thử đầu-cuối tự động trên trình duyệt (Playwright/Cypress), kiểm thử bảo mật

động chuyên sâu (DAST/OWASP ZAP) và kiểm thử tải. Các hạng mục này được ghi nhận trong lộ trình ở Mục 14.

2. Định hướng ITSS và quy trình kiểm thử

Bộ tài liệu được tổ chức bám theo các hoạt động chuẩn của một quy trình kiểm thử: lập kế hoạch, thiết kế testcase, chuẩn bị môi trường, thực thi và thu thập bằng chứng, cuối cùng là đánh giá chất lượng cùng phân tích rủi ro. Mỗi hoạt động được cụ thể hóa bằng các công cụ và sản phẩm thực tế của dự án, như tóm tắt trong Bảng 1.

Table 1: Ánh xạ hoạt động kiểm thử sang sản phẩm thực tế của NaviMart

Hoạt động	Cách thể hiện trong NaviMart
Lập kế hoạch	Xác định mục tiêu, phạm vi, công cụ, môi trường và tiêu chí vào/ra (Mục 3–5).
Thiết kế testcase	Quy ước mã testcase, ma trận truy vết theo loại kiểm thử và phụ lục danh mục chi tiết (Mục 6, Phụ lục A).
Tự động hóa	Jest, ts-jest, @nestjs/testing, supertest, mongodb-memory-server cho backend; Vitest, Testing Library, jsdom cho frontend.
Thực thi & bằng chứng	Kết quả chạy <code>npm run test</code> , <code>npm run test:e2e</code> , <code>vitest run</code> ; báo cáo độ phủ; nhật ký lỗi.
Đánh giá chất lượng	Kết luận theo từng tầng kiểm thử, sổ rủi ro và tiêu chí đóng (Mục 12–13).

3. Tổng quan hệ thống và phạm vi kiểm thử

3.1 Kiến trúc backend

Backend NaviMart được xây dựng trên nền NestJS, cung cấp REST API kèm kênh Web-Socket cho cập nhật thời gian thực. Hệ thống áp dụng xác thực JWT không trạng thái với cặp access/refresh token, và phân quyền hai lớp: theo vai trò (admin/user) ở cấp ứng dụng, và theo quyền của thành viên trong gia đình ở cấp tài nguyên. Dữ liệu được lưu trên MongoDB thông qua Mongoose. Bảng 2 liệt kê các module nghiệp vụ và mức độ được kiểm thử hiện tại; có thể thấy các module cốt lõi (xác thực, gia đình, kho, công thức, bữa ăn, danh sách mua sắm) đều đã được phủ, trong khi một số module ngoại vi vẫn còn khoảng trống.

3.2 Trong và ngoài phạm vi

Trong phạm vi của đợt kiểm thử này là toàn bộ logic service, controller, hàm tiện ích, mapper, guard, khâu kiểm tra dữ liệu và xử lý ngoại lệ ở backend; các kiểm thử đơn

Table 2: Các module backend và mức độ kiểm thử

Module	Nội dung nghiệp vụ	Kiểm thử
auth	Đăng ký, đăng nhập, refresh/rotate token, đăng xuất, quên/đặt lại mật khẩu, xác minh email.	Đầy đủ
families	Tạo gia đình, mã mời, gia nhập, quản lý quyền thành viên.	Đầy đủ
pantry	Quản lý item kho, tiêu thụ/bỏ đi, lọc theo hạn dùng, ghi sự kiện kho.	Đầy đủ
meals	Kế hoạch bữa ăn, gán công thức, tính nguyên liệu thiếu, sinh danh sách mua.	Đầy đủ
recipes	Tìm kiếm/gợi ý theo độ phủ kho, yêu thích, duyệt nội dung, CRUD.	Đầy đủ
shopping-lists	Quản lý danh sách/mục, hoàn tất danh sách và đẩy vào kho.	Đầy đủ
reports	Báo cáo tiêu thụ, lãng phí, chi tiêu (tổng hợp).	Đầy đủ
catalog	Danh mục thực phẩm/nhóm/đơn vị; tìm theo tên/mã vạch.	Đầy đủ
notifications	Thông báo và nhắc hạn dùng.	Đầy đủ
users	Hồ sơ người dùng, cập nhật thông tin cá nhân.	Đầy đủ
admin	Quản trị danh mục/công thức/thống kê/người dùng.	Service (4 service)
realtime	Cổng và dịch vụ WebSocket.	Qua E2E
ai-chef, uploads	Gợi ý món bằng AI; tải ảnh.	Chưa có

vị, kiểm thử API và kiểm thử đầu-cuối; cùng các kiểm thử frontend cho hàm tiện ích, context, component và lớp gọi API. Phân quyền tại các endpoint rủi ro cao (xác thực, gia đình, kho, danh sách mua sắm) được đặc biệt chú trọng. Ngoài phạm vi hiện tại là kiểm thử đầu-cuối trên trình duyệt thật, kiểm thử bảo mật động chuyên sâu, kiểm thử hiệu năng/tải, và các hoạt động vận hành sản xuất như sao lưu/phục hồi.

4. Chiến lược kiểm thử

Chiến lược kiểm thử được phân tầng theo mô hình tháp kiểm thử, với nguyên tắc cốt lõi: *mỗi loại test chỉ chứng minh đúng thứ nó có khả năng chứng minh*. Phân tầng rõ ràng giúp tránh ngộ nhận — ví dụ dùng kiểm thử đơn vị để kết luận về hành vi của cơ sở dữ liệu, hoặc dùng kiểm thử controller độc lập để khẳng định toàn bộ chuỗi bảo mật đã đúng. Bốn tầng backend và một tầng frontend được mô tả trong Bảng 3.

Table 3: Các tầng kiểm thử

Tầng	Mục tiêu	Công cụ & đặc điểm
Unit	Logic service, util, mapper.	Jest với model/service được mock; không khởi tạo toàn bộ Nest context.
API/Controller	Route, mã trạng thái, kiểm tra dữ liệu, hình dạng JSON, ánh xạ ngoại lệ.	supertest qua tiện ích <code>createApiTestApp</code> ; service mock, guard mock.
Security	401/403, quyền, guard, phân quyền cấp đối tượng.	Guard mock ở mức contract; guard <i>thật</i> ở mức E2E.
Integration/E2E	Lưu trữ, luồng xuyên module, realtime.	App NestJS thật + mongodb-memory-server + socket.io-client.
Frontend	Hàm thuần, context, component, lớp API.	Vitest + Testing Library trên jsdom; mock fetch/api/socket.

Bốn nguyên tắc được tuân thủ xuyên suốt. Một là, kiểm thử đơn vị dùng để chứng minh logic nghiệp vụ cục bộ chứ không kết luận về giao dịch hay cơ sở dữ liệu thật. Hai là, kiểm thử API ở tầng controller chứng minh hợp đồng HTTP với cơ chế kiểm tra dữ liệu thật, nhưng do guard và service được mock nên không tự động chứng minh toàn bộ chuỗi bảo mật. Ba là, kiểm thử bảo mật bắt buộc phải bao gồm các đường đi tiêu cực: thiếu token, thiếu quyền, token sai hoặc hết hạn, và truy cập tài nguyên của người khác. Bốn là, kiểm thử đầu-cuối phải dùng nguồn dữ liệu cô lập để kết quả lặp lại được và không phụ thuộc vào cơ sở dữ liệu cục bộ.

Tương ứng với mỗi nhóm, tiêu chí vào/ra được định nghĩa rõ ràng. Nhóm Unit/API/Security yêu cầu mã biên dịch được và spec đã định nghĩa khi bắt đầu, và yêu cầu toàn bộ test pass không còn lỗi không giải thích được khi kết thúc. Nhóm E2E yêu cầu mongodb-memory-server khởi động được và các biến môi trường đã được thiết lập; khi kết thúc, các luồng xuyên module phải được xác minh thành công. Cổng QA cho bản phát hành yêu cầu không còn rủi ro mức P0 nào đang mở, và mọi rủi ro P1 đều có kế hoạch xử lý hoặc được chấp nhận có cân nhắc.

5. Môi trường, công cụ và dữ liệu kiểm thử

Backend được kiểm thử bằng Jest 30 (qua ts-jest 29) kết hợp tiện ích `@nestjs/testing`; các kiểm thử API dùng supertest, các luồng đầu-cuối dùng mongodb-memory-server để cấp một thể hiện MongoDB cô lập ngay trong tiến trình, và socket.io-client cho phần realtime. Frontend dùng Vitest 4.1 trên môi trường jsdom cùng Testing Library. Độ phủ backend được thu thập bằng Jest; độ phủ frontend hiện chưa được cấu hình và được xếp vào công việc của Giai đoạn 2. Tại thời điểm lập tài liệu, dự án *chưa* có pipeline tích hợp liên tục (CI) chạy test tự động — đây là rủi ro được ghi nhận ở Mục 12.

Các lệnh chạy kiểm thử được chuẩn hóa như sau:

```
# Backend (thư mục backend/)
npm run test
npm run test:cov
E2E_USE_MEMORY_MONGO=true npm run test:e2e

# Frontend (thư mục frontend/)
npm run test
```

Về dữ liệu kiểm thử, dự án xây dựng một bộ tiện ích dùng chung đặt ngoài thư mục `src` để không bị tính vào độ phủ. Cụ thể, `fixtures.ts` cung cấp các hàm factory tạo dữ liệu mẫu (người dùng, gia đình, công thức, item kho); `mock-model.ts` mô phỏng model Mongoose với truy vấn nối chuỗi và mặc định trả về rỗng để lỗi “quên mock” lộ ra ngay; còn `api-test-app.ts` dựng nhanh một controller trong app Nest với guard và pipe có thể cấu hình trạng thái xác thực/quyền theo từng test. Ở frontend, việc mock được thực hiện trực tiếp bằng `vi.mock` cho lớp gọi API và socket. Riêng kiểm thử đầu-cuối dùng dữ liệu được seed thật vào MongoDB cô lập và được dọn dẹp sau mỗi bộ test.

6. Thiết kế testcase và ma trận truy vết

Testcase được thiết kế dựa trên rủi ro nghiệp vụ, luồng chính, luồng lỗi, kiểm tra dữ liệu, phân quyền, chuyển trạng thái và tính nhất quán dữ liệu. Để bảo đảm khả năng truy vết, mỗi testcase được gán một mã có tiền tố theo module và theo loại kiểm thử, theo quy ước trong Bảng 4. Các kiểm thử API hiện có trong mã nguồn đã áp dụng sẵn quy ước này (ví dụ API-AUTH-001, API-PAN-005).

Table 4: Quy ước mã testcase

Tiền tố	Ý nghĩa	Ví dụ
API-<MOD>	Kiểm thử API/controller theo module.	API-FAM-003
UT-<MOD>	Kiểm thử đơn vị service/util.	UT-PAN-009
SEC-*	Kiểm thử bảo mật/phân quyền.	SEC-JWT-001
E2E-*, INT-*	Kiểm thử đầu-cuối / tích hợp.	INT-CONC-001
FE-*	Kiểm thử frontend.	FE-AUTH-001

Ma trận truy vết (Bảng 5) cho thấy mỗi module được phủ bởi những loại kiểm thử nào. Có thể đọc ma trận này như một bản đồ độ tin cậy: các ô đầy đủ cho biết nghiệp vụ đã được chứng minh ở nhiều tầng, trong khi các ô trống chỉ ra nơi cần ưu tiên bổ sung.

Về kỹ thuật thiết kế, dự án vận dụng phân hoạch tương đương và phân tích giá trị biên cho các trường số lượng và thời hạn (ví dụ số lượng bằng/lớn hơn/nhỏ hơn tồn kho; cửa sổ nhắc hạn bằng 0 ngày), kiểm thử chuyển trạng thái (item kho chuyển giữa các trạng thái hoạt động/đã dùng hết; danh sách mua chuyển sang hoàn tất), kiểm thử tiêu cực (không tìm thấy, hoàn tất danh sách đã hoàn tất, sai vai trò), ma trận RBAC cho phân quyền, và kiểm thử tương tranh cho điều kiện đua (race condition) khi tiêu thụ kho đồng thời.

Table 5: Ma trận truy vết theo loại kiểm thử

Module	Unit	API	Security	Integration/E2E
auth	service (21)	API (10)	401; JWT âm tính (SEC-JWT-001..005)	đăng ký/refresh/đặt lại mật khẩu
families	service (13), util (4)	API (7)	401/403	mời/gia nhập
pantry	service (15), util (11)	API (8)	401/403; object-level (SEC-OBJ-001)	lọc hạn dùng; concurrency (INT-CONC-001)
recipes	service (19)	API (7)	401/403 (sai vai trò)	yêu thích/gợi ý
shopping-lists	service (12)	API (6)	401/403; object-level (SEC-OBJ-002)	hoàn tất → kho
meals	service (24)	API (8)	401/403; object-level (SEC-OBJ-003)	bữa ăn → thiếu → danh sách
reports	service (3)	API (6)	401/403 (view_reports)	tổng hợp báo cáo
catalog	service (6)	API (5)	401	đọc qua E2E
users	service (7)	API (6)	401	hồ sơ qua E2E
notifications	service (8) + nhắc hạn (6)	API (5)	401	một phần
admin	4 service (39)	—	promote (e2e)	nâng quyền admin (e2e)

7. Kiểm thử đơn vị

Kiểm thử đơn vị tập trung vào logic nghiệp vụ ở mức service và hàm tiện ích, với mọi phụ thuộc (model Mongoose, service khác) đều được mock để test nhanh, cô lập và dễ truy nguyên lỗi. Đây là tầng mang lại lợi tức cao nhất bởi nó kiểm soát trực tiếp các quy tắc nghiệp vụ và các nhánh rẽ phức tạp.

Nhóm **xác thực** được xem là ưu tiên cao nhất do liên quan trực tiếp đến danh tính và token. Hai mươi một testcase bao phủ trọn vẹn vòng đời token và mật khẩu: tạo người dùng kèm gia đình sở hữu rồi phát token; chuyển lỗi trùng khóa thành xung đột; từ chối khi sai định danh, sai mật khẩu hoặc tài khoản chưa kích hoạt; xác minh và xoay vòng refresh token (bao gồm trường hợp sai chữ ký và hash không khớp); đặt lại mật khẩu và xác minh email với token hợp lệ, sai và hết hạn; không tiết lộ sự tồn tại của tài khoản khi quên mật khẩu; và cấp token tạm trong môi trường không phải sản xuất khi tắt gửi email.

Nhóm **khó thực phẩm** kết hợp mười lăm testcase cho service và mười một testcase cho hàm tiện ích tính hạn dùng. Phần service kiểm tra việc lọc theo gia đình, từ chối tiêu thụ item không hoạt động hoặc vượt tồn, chuyển sang trạng thái đã dùng hết khi tiêu thụ hết kèm ghi sự kiện, tái kích hoạt khi số lượng dương trở lại, thay thế thuộc tính từ danh mục khi đổi loại thực phẩm, và đặc biệt là *trừ kho nguyên tử* để chống điều kiện đua (trình bày chi tiết tại Mục 10.4). Phần hàm tiện ích kiểm tra việc chuẩn hóa thời gian về nửa đêm, cộng ngày mà không làm thay đổi đầu vào, và phân loại trạng thái an toàn/sắp hết hạn/đã hết hạn theo cửa sổ cảnh báo, bao gồm cả giá trị biên.

Các nhóm nghiệp vụ còn lại được phủ tương tự. Công thức (19 testcase) kiểm tra xếp hạng gợi ý theo tỷ lệ khớp nguyên liệu, ưu tiên item sắp hết hạn, đếm lượt yêu thích, tự duyệt khi người tạo là admin và cấm người dùng thường sửa công thức của người khác. Bữa ăn (24 testcase gồm cả tính nguyên liệu thiếu và sinh danh sách mua) kiểm tra việc khớp nguyên liệu theo mã/đơn vị/tên, co giãn theo khẩu phần và bỏ qua nguyên liệu tùy chọn. Danh sách mua sắm (12 testcase) kiểm tra việc hoàn tất danh sách và đẩy các mục đã mua vào kho, từ chối hoàn tất danh sách đã hoàn tất hoặc không có mục nào được tích. Gia đình (13+4 testcase) kiểm tra quy tắc phân quyền nội bộ như cấm tự sửa quyền của chính mình hay sửa quyền của chủ gia đình. Các module được bổ sung kiểm thử trong đợt này — danh mục, người dùng, thông báo, nhắc hạn (expiry) và bốn service quản trị (thống kê, danh mục, công thức, người dùng) — lấp các khoảng trống độ phủ rõ rệt nhất, đưa độ phủ tổng từ 56% lên 68% câu lệnh.

8. Kiểm thử API/Controller

Kiểm thử API xác minh hợp đồng HTTP ở tầng controller: đường dẫn, phương thức, tham số đường dẫn và truy vấn, thân yêu cầu, khâu kiểm tra dữ liệu (loại bỏ trường lạ, ép kiểu), hình dạng phản hồi và việc ánh xạ ngoại lệ thành mã trạng thái phù hợp. Các kiểm thử dùng tiện ích `createApiTestApp` để dựng controller trong một app Nest với guard mock có thể cấu hình trạng thái xác thực và quyền theo từng kịch bản. Bảng 6 tóm tắt 68 testcase API trên mười module.

Một điểm đáng chú ý về mặt phương pháp: trong mọi kịch bản tiêu cực, kiểm thử không chỉ khẳng định mã trạng thái trả về đúng, mà còn khẳng định service *không* được gọi khi yêu cầu bị từ chối. Điều này bảo đảm lớp kiểm tra dữ liệu và phân quyền chặn yêu cầu sai *trước khi* chạm tới logic nghiệp vụ.

9. Kiểm thử bảo mật

Kiểm thử bảo mật tập trung vào xác thực và phân quyền tại các endpoint rủi ro cao. Ở tầng API, guard được mock để xác minh việc ánh xạ 401/403 và nguyên tắc “service không được gọi khi bị từ chối”. Tuy nhiên, để chứng minh chuỗi bảo mật thực sự hoạt động, đợt kiểm thử này bổ sung một bộ kiểm thử đầu-cuối chạy qua **chiến lược Passport JWT và guard thật** — không mock — trong tệp `test/security.e2e-spec.ts`. Các token âm tính (sai chữ ký, hết hạn) được tạo trực tiếp bằng thư viện `jsonwebtoken` ngay trong test.

Cần phân biệt hai lớp kiểm thử bảo mật để không hiểu nhầm độ sâu. Ở *tầng API* (guard

mock), mỗi module chỉ có thể chứng minh hai điều mà một guard giả lập kiểm soát được: chưa xác thực → 401 và thiếu quyền → 403; vì vậy cột Security của nhiều module trong ma trận chủ yếu là 401/403. Các bảo đảm sâu hơn — token giả mạo/hết hạn và phân quyền cấp đối tượng — không thể chứng minh bằng guard mock mà phải chạy qua *guard thật* trên dữ liệu thật, nên nằm ở tầng E2E.

Nhóm kiểm thử SEC-OBJ minh họa phân quyền cấp đối tượng trên ba loại tài nguyên thuộc gia đình (item kho, danh sách mua, kế hoạch bữa ăn): cùng một tài nguyên, chủ sở hữu truy cập được (200) nhưng người dùng thuộc gia đình khác nhận về 404 thay vì 403 — nghĩa là hệ thống không những từ chối truy cập mà còn không tiết lộ sự tồn tại của tài nguyên, tránh rò rỉ thông tin qua mã trạng thái. Hạng mục duy nhất chưa thực hiện là quét bảo mật động (OWASP ZAP), được ghi nhận là nằm ngoài phạm vi hiện tại do chưa có hạ tầng triển khai.

10. Kiểm thử tích hợp và đầu-cuối

Kiểm thử đầu-cuối nhằm xác minh những hành vi không thể chứng minh bằng mock: tính bền vững của dữ liệu, các luồng nghiệp vụ xuyên module, hoạt động thực của guard và JWT, cùng cập nhật realtime qua WebSocket. Toàn bộ chạy trên một app NestJS thật với MongoDB cô lập do mongodb-memory-server cung cấp.

Hai bộ luồng chính đã được xây dựng. Bộ **main-flows** bao gồm khoảng mười luồng nền tảng: đăng ký và làm mới token, tạo mã mời và gia nhập gia đình bằng người dùng thứ hai, hoàn tất danh sách mua để đẩy hàng vào kho, lọc kho theo trạng thái hạn dùng, và chuỗi nghiệp vụ từ bữa ăn qua nguyên liệu thiếu đến danh sách mua và báo cáo. Bộ **extended-flows** bổ sung khoảng mười bốn luồng nâng cao như nâng quyền quản trị, truy vấn danh mục, đồng bộ lượt yêu thích, gợi ý theo độ phủ kho, đặt lại mật khẩu và cập nhật realtime.

10.1 Chuẩn hóa môi trường thiết lập

Các bộ test đầu-cuối đều thiết lập biến môi trường (khóa JWT, cấu hình MongoDB in-memory, lịch chạy tác vụ định kỳ) *trước khi* nạp module ứng dụng, bởi cấu hình được chốt tại thời điểm nạp. Một trở ngại được phát hiện và xử lý trong đợt này: phụ thuộc **cloudinary** (chỉ cần lúc nạp module tải ảnh) chưa được cài đặt khiến toàn bộ E2E không khởi động được. Giải pháp là cung cấp một stub ngoại tuyến tại `test/stubs/cloudinary.ts` và ánh xạ qua `moduleNameMapper` trong cấu hình Jest E2E, nhờ đó ứng dụng nạp được mà không cần thư viện thật và không phụ thuộc mạng.

10.2 Case study: phát hiện và sửa lỗi trừ kho vượt mức (oversell)

Đáng chú ý nhất trong đợt kiểm thử là một lỗi tương tranh thực sự được phát hiện nhờ một testcase được thiết kế có chủ đích. Testcase **INT-CONC-001** bắn *năm* yêu cầu tiêu thụ đồng thời vào cùng một item kho có tồn 2 đơn vị, rồi kiểm tra bất biến “không trừ vượt”: chỉ đúng một yêu cầu được phép thành công, tồn kho cuối phải bằng 0 và không bao giờ âm.

Khi chạy lần đầu, testcase thất bại với kết quả cả *năm* yêu cầu đều thành công. Nguyên nhân nằm ở mẫu *đọc-sửa-ghi* cổ điển trong hàm tiêu thụ: hàm đọc item, kiểm tra số lượng, trừ đi rồi lưu lại. Dưới tải đồng thời, cả năm yêu cầu cùng đọc thấy tồn kho là 2, cùng vượt qua bước kiểm tra, rồi cùng ghi đè — dẫn đến trừ kho vượt mức (oversell).

Lỗi đã được sửa bằng cách chuyển sang một thao tác *cập nhật nguyên tử* có điều kiện: dùng `findOneAndUpdate` với bộ lọc bảo vệ “số lượng phải lớn hơn hoặc bằng lượng cần trừ” kết hợp một aggregation pipeline để vừa trừ vừa cập nhật trạng thái trong một bước. Khi tồn kho không đủ cho hai yêu cầu, chỉ một yêu cầu khớp bộ lọc và thắng cuộc; các yêu cầu còn lại không khớp và nhận về lỗi 400 “vượt tồn kho”. Sau khi sửa, testcase cho kết quả đúng như kỳ vọng: một phản hồi 200 và bốn phản hồi 400. Lỗi này được ghi nhận là BUG-CONC-001 và việc khắc phục thỏa mãn tiêu chí đóng của rủi ro TEST-004.

11. Kết quả thực thi và độ phủ

11.1 Kết quả regression

Toàn bộ bộ kiểm thử ở trạng thái xanh. Backend chạy 264 testcase đơn vị/API trên 34 bộ test cùng 32 testcase đầu-cuối trên 4 bộ test, tất cả đều pass, không có lỗi hay thất bại nào; thời gian chạy phần đơn vị/API khoảng mười giây. Frontend chạy 61 testcase trên 9 tệp, cũng pass toàn bộ. So với hiện trạng đầu kỳ (176 testcase đơn vị/API backend và 32 testcase frontend), bộ test đã được mở rộng đáng kể: bổ sung kiểm thử đơn vị cho các service trước đây bỏ trống (danh mục, người dùng, thông báo, nhắc hạn, ba service quản trị), bổ sung kiểm thử API ở mức HTTP cho năm controller (danh mục, người dùng, thông báo, bữa ăn, báo cáo), thêm kiểm thử bảo mật và tương tranh ở backend, cùng các kiểm thử frontend mới — đồng thời phát hiện và sửa một lỗi nghiệp vụ thực.

11.2 Độ phủ mã nguồn backend

Độ phủ tổng thể trên toàn bộ `src` đạt 68,03% câu lệnh và 57,97% nhánh, tăng đáng kể so với mức 56,24% / 47,38% đầu đợt nhờ việc lấp các module trước đây bỏ trống. Phần còn lại kéo tỷ lệ chung xuống chủ yếu là các tệp chưa được phủ (ai-chef, uploads, cổng realtime, controller quản trị, cấu hình, seed dữ liệu) — nhiều trong số đó phù hợp để phủ bằng kiểm thử đầu-cuối hơn là kiểm thử đơn vị. Khi xét riêng theo module (Bảng 9), các module nghiệp vụ cốt lõi đều có độ phủ cao — phản ánh đúng chiến lược “ưu tiên logic nghiệp vụ”.

Độ phủ frontend hiện chưa đo được do Vitest chưa được cấu hình thu thập coverage; việc bật công cụ đo được xếp vào Giai đoạn 2.

11.3 Phân tích lý do

Phân bố độ phủ phản ánh ba lựa chọn có chủ đích. Thứ nhất, kiểm thử đơn vị được dồn vào tầng service — nơi chứa hầu hết quy tắc nghiệp vụ và nhánh rẽ — nên các module cốt lõi đạt độ phủ cao, khóa chặt rủi ro sai lệch nghiệp vụ. Thứ hai, việc cố ép độ phủ cho

các tệp cấu hình, seed, cổng realtime hay DTO mang lại lợi tức thấp về phát hiện lỗi và làm tăng chi phí bảo trì, nên những phần này được ưu tiên phủ bằng kiểm thử đầu-cuối thay vì kiểm thử đơn vị. Thứ ba, các khoảng trống còn lại (admin, uploads, ai-chef) được xác định rõ là hạng mục ưu tiên khi thiết lập cổng chất lượng tăng dần trong các giai đoạn sau.

12. Rủi ro, lỗi và tiêu chí đóng

12.1 Số rủi ro

Sau đợt kiểm thử, hai rủi ro mức P0 quan trọng nhất đã được đóng: lỗi tương tranh trừ kho và sự thiếu hụt kiểm thử bảo mật âm tính. Các rủi ro còn lại chủ yếu thuộc về hạ tầng kiểm thử — thiếu CI, thiếu cổng độ phủ, frontend chưa đo coverage — và được quy hoạch vào Giai đoạn 2 (Bảng 10).

12.2 Lỗi đã ghi nhận

Đợt kiểm thử ghi nhận một lỗi nghiệp vụ thực và đã khắc phục: **BUG-CONC-001** — hàm tiêu thụ kho theo mẫu đọc-sửa-ghi gây trừ vượt khi tiêu thụ đồng thời, đã sửa bằng cập nhật nguyên tử có điều kiện và được bảo vệ bởi INT-CONC-001 cùng một kiểm thử đơn vị tương ứng. Hai mục còn lại liên quan đến hạ tầng (thiếu pipeline CI và thiếu cổng độ phủ) được chuyển sang Giai đoạn 2.

12.3 Tiêu chí đóng P0

Hai tiêu chí P0 liên quan đến chất lượng nghiệp vụ và bảo mật đã được thỏa mãn: kiểm thử tương tranh chứng minh hệ thống chống trừ vượt, và kiểm thử bảo mật âm tính (JWT, phân quyền cấp đối tượng) đã có bằng chứng. Hai tiêu chí còn lại — thiết lập CI chạy test cho cả backend lẫn frontend, và thiết lập cổng độ phủ chống suy giảm — thuộc về Giai đoạn 2.

13. Đánh giá mức độ đáp ứng và kết luận

13.1 Đánh giá theo từng tầng

Xét theo từng tầng kiểm thử, có thể đánh giá như sau. Kiểm thử đơn vị ở mức *tốt*: các service cốt lõi được phủ cao với bộ fixture và mock chuẩn hóa. Kiểm thử API cũng ở mức *tốt*: hợp đồng được kiểm chứng đầy đủ cho năm module chính và có mã truy vết rõ ràng. Kiểm thử bảo mật đạt mức *tốt* sau khi bổ sung các đường đi âm tính qua guard thật. Kiểm thử tích hợp/đầu-cuối ở mức *tốt* với các luồng xuyên module, realtime và tương tranh chạy trên cơ sở dữ liệu cô lập. Riêng hạng mục độ phủ và CI *cần cải thiện* và là trọng tâm của Giai đoạn 2.

13.2 Kết luận

Bộ kiểm thử NaviMart đã có nền tảng vững cho regression ở tầng service và controller, đồng thời sở hữu một lớp kiểm thử đầu-cuối thực sự với MongoDB cô lập và WebSocket. Toàn bộ kiểm thử ở trạng thái xanh và quá trình kiểm thử đã chứng tỏ giá trị thực tiễn khi phát hiện rồi khắc phục một lỗi tương tranh nghiêm trọng. Tuy vậy, do phần hạ tầng kiểm thử (tích hợp liên tục, cổng độ phủ và đo độ phủ frontend) chưa hoàn tất, hệ thống chưa nên được coi là đạt QA toàn diện cho bản phát hành; những hạng mục này được chủ động hoãn sang Giai đoạn 2 theo quyết định của nhóm.

13.3 Lộ trình cải tiến

Bảng 11 tổng hợp lộ trình. Các hạng mục Giai đoạn 1 đã hoàn thành trong đợt này; các hạng mục Giai đoạn 2 (hạ tầng) được hoãn lại theo kế hoạch.

A. Phụ lục A. Danh mục testcase chi tiết

A.1 Backend — kiểm thử đơn vị

Tệp	Nội dung (số testcase)
<code>auth.service.spec.ts</code>	Vòng đời token và mật khẩu: tạo người dùng + gia đình, phát/xoay token, đặt lại và xác minh email, không lộ tồn tại tài khoản (21).
<code>families.service.spec.ts</code>	Tạo gia đình, gia nhập theo mã mời, các quy tắc phân quyền nội bộ (13).
<code>family-access.util.spec.ts</code>	Từ chối khi không hoạt động/không phải thành viên/đã rời (4).
<code>pantry.service.spec.ts</code>	CRUD kho, tiêu thụ/bỏ đi, sự kiện kho, và trừ kho nguyên tử chống tương tranh (15).
<code>expiry-status.util.spec.ts</code>	Tính và phân loại hạn dùng, gồm giá trị biên (11).
<code>meals.service.spec.ts</code>	Kế hoạch bữa ăn, gán công thức, ủy quyền tính nguyên liệu thiếu (11).
<code>missing-ingredients.service.spec.ts</code>	Khớp nguyên liệu, co giãn theo khẩu phần, xử lý nguyên liệu tùy chọn (10).
<code>shopping-list-generation.service.spec.ts</code>	Sinh danh sách từ nguyên liệu thiếu (4).
<code>recipes.service.spec.ts</code>	Xếp hạng gợi ý, yêu thích, duyệt nội dung, phân quyền chỉnh sửa (19).

Tệp	Nội dung (số testcase)
<code>shopping-lists.service.spec.ts</code>	Quản lý danh sách, hoàn tất và đẩy vào kho (12).
<code>reports.service.spec.ts</code>	Tổng hợp tiêu thụ/lãng phí/bảng điều khiển (3).
<code>catalog.service.spec.ts</code>	Danh mục, tìm theo tên/mã vạch, ánh xạ phản hồi (6).
<code>users.service.spec.ts</code>	Hồ sơ, cập nhật thông tin và thiết lập thông báo (7).
<code>notifications.service.spec.ts</code>	Liệt kê, đánh dấu đã đọc, tạo có khứ trùng lặp (8).
<code>expiry-notifications.service.spec.ts</code>	Quét item hết hạn, phân loại expired/expiring/safe, dedupe, phát realtime (6).
<code>admin-stats.service.spec.ts</code>	Tổng hợp thống kê người dùng/gia đình/công thức (2).
<code>admin-recipes.service.spec.ts</code>	Phân trang kiểm duyệt, cập nhật status, chặn công thức đã lưu trữ (8).
<code>admin-users.service.spec.ts</code>	Lọc/tìm kiếm, tạo (băm mật khẩu, Conflict), cập nhật, xóa mềm (12).
<code>admin-catalog.service.spec.ts</code>	CRUD danh mục/món/đơn vị: slugify, kiểm tra category, Conflict, xóa mềm (13).

A.2 Backend — kiểm thử API, đầu-cuối và controller

Controller (mock service): `auth` (4), `families` (3), `pantry` (4), `recipes` (4).

API (supertest, 68): `API-AUTH-001..010`, `API-FAM-001..007`, `API-PAN-001..008`, `API-RCP-001..007`, `API-SL-001..006`, `API-MEAL-001..008`, `API-RPT-001..006`, `API-CAT-001..005`, `API-USR-001..006`, `API-NTF-001..005`.

Đầu-cuối: `main-flows` (~10 luồng), `extended-flows` (~14 luồng), `security` (`SEC-JWT-001..005` và `SEC-OBJ-000..003`, 9 test), `concurrency` (`INT-CONC-001`, 1 test).

A.3 Frontend (Vitest)

Tệp	Nội dung (số testcase)
<code>utils/expiry.test.ts</code>	Tính số ngày tới hạn/còn lại/quá hạn (8).
<code>api/client.test.ts</code>	Lưu/đọc token, làm mới khi 401, phát sự kiện chưa xác thực (9).
<code>components/ProtectedRoute.test.tsx</code>	Điều hướng theo trạng thái đăng nhập và vai trò (4).

Tệp	Nội dung (số testcase)
<code>components/FoodAutocomplete.test.tsx</code>	Gợi ý có trễ và chọn món (2).
<code>contexts/AuthContext.test.tsx</code>	Đăng nhập/đăng ký/đăng xuất, làm mới phiên (6).
<code>contexts/DialogContext.test.tsx</code>	Hộp thoại cảnh báo và xác nhận (5).
<code>api/index.test.ts</code>	Lớp gọi API (auth/pantry/recipes/shopping-lists/catalog/users/family/notifications) ánh xạ đúng đường dẫn và tùy chọn (25).
<code>components/Button.test.tsx</code>	Render children, bắt onClick, vô hiệu khi disabled, hiển thị icon (4).
<code>pages/Login.test.tsx</code>	Kiểm tra dữ liệu, điều hướng theo vai trò, hiển thị lỗi (4).

B. Phụ lục B. Checklist chạy và nghiệm thu

Bước	Lệnh / hoạt động	Kết quả mong đợi
1	<code>cd backend && npm run test</code>	34 bộ / 264 testcase pass.
2	<code>cd backend && npm run test:cov</code>	Sinh báo cáo độ phủ trong <code>backend/coverage</code> .
3	<code>E2E_USE_MEMORY_MONGO=true npm run test:e2e</code>	4 bộ / 32 testcase đầu-cuối pass.
4	<code>cd frontend && npm run test</code>	9 tệp / 61 testcase pass.
5	(Giai đoạn 2) Kiểm tra pipeline CI	Job backend và frontend chạy test/độ phủ thành công.
6	Rà soát sổ rủi ro	Không còn rủi ro P0 đang mở.

Table 6: Danh mục kiểm thử API

Module	Mã	Kịch bản tiêu biểu
auth	API-AUTH-001..010	Đăng ký 201; thiếu mật khẩu/trường lạ 400; trùng email/sđt 409; đăng nhập 200; sai thông tin 401; đăng xuất không token 401.
families	API-FAM-001..007	Lấy gia đình 200; không có gia đình 404; tạo mời thiếu quyền 403; mã mời sai 404; đã là thành viên 409; tự sửa quyền 400.
pantry	API-PAN-001..008	Liệt kê 200; chưa đăng nhập 401; tạo thiếu quyền 403; số lượng âm 400; tiêu thụ quá tồn 400; không tồn tại 404.
recipes	API-RCP-001..007	Tìm kiếm/gợi ý 200; công thức đã lưu trữ 404; tạo sai vai trò 403; thiếu tên 400; thêm yêu thích 201.
shopping-lists	API-SL-001..006	Tạo 201; thiếu quyền 403; thiếu số lượng 400; danh sách không tồn tại 404; hoàn tất danh sách đã hoàn tất 400; hoàn tất hợp lệ 200.
meals	API-MEAL-001..008	Liệt kê theo khoảng ngày 200; thiếu ngày 400; tạo thiếu quyền 403; thiếu session 400; tạo hợp lệ 201; không tồn tại 404.
reports	API-RPT-001..006	Dashboard/xu hướng/lãng phí 200; chưa đăng nhập 401; thiếu quyền view_reports 403; thiếu khoảng ngày 400.
catalog	API-CAT-001..005	Danh mục/đơn vị 200; chưa đăng nhập 401; tìm món kèm query 200; limit ngoài khoảng 400.
users	API-USR-001..006	Lấy hồ sơ 200; chưa đăng nhập 401; cập nhật hợp lệ 200; gender sai enum 400; trường lạ 400; không tồn tại 404.
notifications	API-NTF-001..005	Liệt kê 200; chưa đăng nhập 401; limit ngoài khoảng 400; đánh dấu đã đọc/đọc tất cả 200.

Table 7: Danh mục kiểm thử bảo mật

Mã	Mục tiêu	Trạng thái
SEC-JWT-001	Không có header xác thực → 401 (guard thật).	Pass
SEC-JWT-002	Token dị dạng → 401.	Pass
SEC-JWT-003	Token ký sai khóa bí mật → 401.	Pass
SEC-JWT-004	Token hết hạn (đúng khóa) → 401.	Pass
SEC-JWT-005	Token hợp lệ nhưng người dùng không tồn tại → 401.	Pass
SEC-OBJ-000	Chủ sở hữu đọc tài nguyên của chính mình → 200.	Pass
SEC-OBJ-001	Gia đình B đọc <i>item kho</i> của gia đình A → 404 (không lộ dữ liệu).	Pass
SEC-OBJ-002	Gia đình B đọc <i>danh sách mua</i> của gia đình A → 404.	Pass
SEC-OBJ-003	Gia đình B đọc <i>kế hoạch bữa ăn</i> của gia đình A → 404.	Pass
SEC-DAST-001	Quét OWASP ZAP baseline.	Ngoài phạm vi

Table 8: Tổng hợp kết quả regression

Phạm vi	Bộ test	Testcase (pass/tổng)
Backend đơn vị/API	34	264 / 264
Backend đầu-cuối	4	32 / 32
Frontend (Vitest)	9	61 / 61

Table 9: Độ phủ câu lệnh theo module (backend)

Module	% câu lệnh	Nhận xét
notifications	88,11	Đạt (service + API + nhắc hạn)
meals	85,36	Đạt
families	84,76	Đạt
users	83,33	Đạt (service + API)
pantry	83,04	Đạt
catalog	82,14	Đạt (service + API)
reports	81,48	Đạt (service + API)
recipes	78,81	Đạt
shopping-lists	76,81	Đạt
auth	71,05	Đạt (service cao; controller/strategy kéo xuống)
admin	47,54	Trung bình (service đã test; controller chưa)
realtime	9,75	Thấp (chỉ qua E2E)
ai-chef, uploads	0,00	Chưa có kiểm thử đơn vị

Table 10: Sổ rủi ro kiểm thử

ID	Mức	Mô tả / ảnh hưởng	Trạng thái
TEST-004	High	Lỗi tương tranh trừ kho gây oversell.	Đã đóng
TEST-005	Med	Thiếu kiểm thử JWT âm tính và phân quyền cấp đối tượng.	Đã đóng
TEST-003	Med	Còn vài phần chưa có kiểm thử (ai-chef, uploads, cổng realtime, controller quản trị).	Giảm thiểu phần lớn
TEST-001	High	Chưa có CI chạy test $\Rightarrow$ PR lỗi vẫn có thể gộp.	Mở (Giai đoạn 2)
TEST-002	Med	Frontend chưa cấu hình độ phủ.	Mở (Giai đoạn 2)
TEST-006	Med	Chưa có cổng chất lượng độ phủ.	Mở (Giai đoạn 2)
TEST-007	Low	Mock frontend gắn chặt cài đặt (chưa dùng MSW).	Theo dõi

Table 11: Lộ trình cải tiến kiểm thử

Giai đoạn	Ưu tiên	Hạng mục
1	P0	Kiểm thử tương tranh và sửa lỗi oversell — <i>đã làm</i> .
1	P1	Kiểm thử JWT âm tính và phân quyền cấp đối tượng qua guard thật — <i>đã làm</i> .
1	P1	Bổ sung kiểm thử cho danh mục, người dùng, thông báo, thống kê quản trị — <i>đã làm</i> .
1	P2	Mở rộng kiểm thử frontend (lớp API, trang chính) — <i>đã làm</i> .
2	P0	Thiết lập pipeline CI chạy test backend và frontend.
2	P0	Bật đo độ phủ frontend và đặt cổng độ phủ cho cả hai phía.
—	P2	Bổ sung E2E trình duyệt (Playwright) và quét DAST (OWASP ZAP).